CASE TÉCNICO

Analista de Integração de Dados — Implantação Systock

DOCUMENTAÇÃO TÉCNICA — PROCESSO DE INTEGRAÇÃO E TRANSFORMAÇÃO DE DADOS

---

1. Introdução

Este documento tem como objetivo descrever o processo de **extração, transformação e carga de dados (ETL)** realizado a partir de uma base fornecida em formato de planilha, com posterior persistência em banco de dados relacional PostgreSQL.

O trabalho contempla desde a análise inicial dos dados até a implementação de regras de transformação, validação de integridade e consultas analíticas, simulando um cenário real de integração de dados em ambiente corporativo.

---

2. Análise Inicial da Base de Dados

Antes da execução do processo de carga, foi realizada uma análise detalhada da estrutura dos dados fornecidos. Durante essa etapa, foram identificadas inconsistências estruturais e problemas de qualidade de dados que poderiam comprometer a integridade das informações no banco de dados.

Entre os principais pontos identificados, destacam-se:

- Existência de colunas duplicadas ou com nomenclatura inconsistente, como o caso da coluna "produto_id " contendo espaço em branco ao final do nome;

- Divergência entre nomes de campos utilizados nas tabelas e aqueles presentes nos dados de origem;

- Presença de registros com valores inválidos ou inconsistentes, como ordem_compra = 0, dificultando o rastreamento de pedidos;

- Estrutura incorreta de chave primária em algumas tabelas, incluindo referências a colunas inexistentes;

- Dados financeiros inconsistentes, como registros onde o preço de compra era superior ao preço de venda.

Essa análise foi essencial para definir as regras de tratamento a serem aplicadas na etapa de transformação.

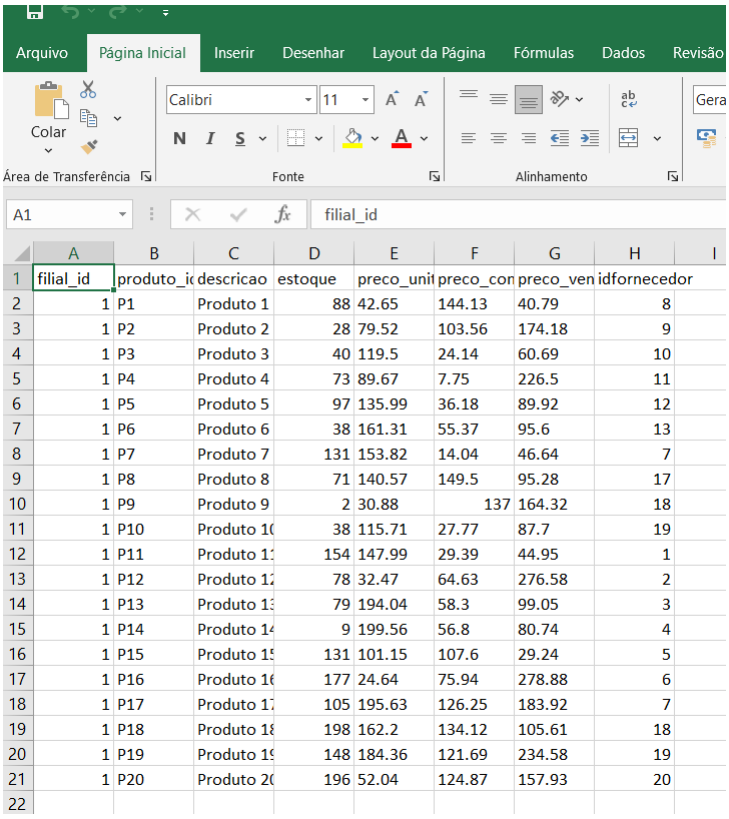
Banco Corrigido

```sql
CREATE TABLE public.fornecedor(
    idfornecedor int4 NOT NULL,
    razao_social varchar(255) NOT NULL,
    CONSTRAINT fornecedor_pkey PRIMARY KEY (idfornecedor)
);

CREATE TABLE public.produtos_filial(
    filial_id int4 NOT NULL,
    produto_id varchar(255) NOT NULL,
    descricao varchar(255) NOT NULL,
    estoque float8 DEFAULT 0 NOT NULL,
    preco_unitario float8 DEFAULT 0 NOT NULL,
    preco_compra float8 DEFAULT 0 NOT NULL,
    preco_venda float8 DEFAULT 0 NOT NULL,
    idfornecedor int4,
    CONSTRAINT produtos_filial_pkey PRIMARY KEY (filial_id, produto_id),
    CONSTRAINT fk_produto_fornecedor
        FOREIGN KEY (idfornecedor)
        REFERENCES fornecedor(idfornecedor)
);

CREATE TABLE public.venda(
    venda_id int8 NOT NULL,
    data_emissao date NOT NULL,
    horariomov varchar(8) DEFAULT '00:00:00' NOT NULL,
    produto_id varchar(25) NOT NULL,
    qtde_vendida float8,
    valor_unitario numeric(12, 4) DEFAULT 0 NOT NULL,
    filial_id int8 DEFAULT 1 NOT NULL,
    item int4 DEFAULT 0 NOT NULL,
    unidade_medida varchar(3),
    CONSTRAINT pk_venda PRIMARY KEY
        (filial_id, venda_id, data_emissao, produto_id, item, horariomov)
);

CREATE TABLE public.pedido_compra(
    pedido_id float8 NOT NULL,
    data_pedido date,
    item float8 NOT NULL,
    produto_id varchar(25) NOT NULL,
    descricao_produto varchar(255),
    ordem_compra float8 NOT NULL,
    qtde_pedida float8,
    filial_id int4,
    data_entrega date,
    qtde_entregue float8 DEFAULT 0 NOT NULL,
    qtde_pendente float8 DEFAULT 0 NOT NULL,
    preco_compra float8 DEFAULT 0,
    fornecedor_id int4,
    CONSTRAINT pedido_compra_pkey
        PRIMARY KEY (pedido_id, produto_id, item)
);
```

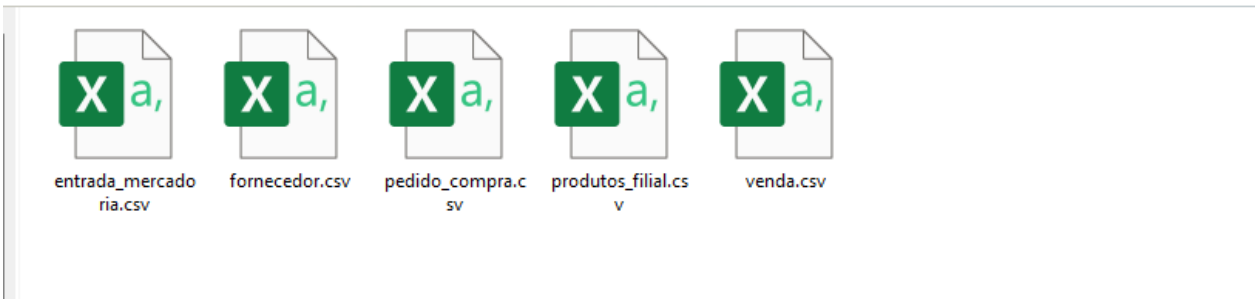
```sql
CREATE TABLE public.entradas_mercadoria (
    ordem_compra float8 NOT NULL,
    data_entrada date,
    nro_nfe varchar(255) NOT NULL,
    item float8 NOT NULL,
    produto_id varchar(25) NOT NULL,
    descricao_produto varchar(255),
    qtde_recebida float8,
    filial_id int4,
    custo_unitario numeric(12, 4) DEFAULT 0 NOT NULL,
    CONSTRAINT entradas_mercadoria_pkey
        PRIMARY KEY (ordem_compra, item, produto_id, nro_nfe)
);

-- pedido_compra -> fornecedor
ALTER TABLE pedido_compra
ADD CONSTRAINT fk_pedido_fornecedor
FOREIGN KEY (fornecedor_id)
REFERENCES fornecedor(idfornecedor);
```

3. Processo de ETL

O processo de ETL foi dividido em três etapas principais: **extração, transformação e carga**, seguindo boas práticas de integração de dados.



filial_id	produto_id	descricao	estoque	preco_unit	preco_con	preco_ven	idfornecedor
1	P1	Produto 1	88	42.65	144.13	40.79	8
1	P2	Produto 2	28	79.52	103.56	174.18	9
1	P3	Produto 3	40	119.5	24.14	60.69	10
1	P4	Produto 4	73	89.67	7.75	226.5	11
1	P5	Produto 5	97	135.99	36.18	89.92	12
1	P6	Produto 6	38	161.31	55.37	95.6	13
1	P7	Produto 7	131	153.82	14.04	46.64	7
1	P8	Produto 8	71	140.57	149.5	95.28	17
1	P9	Produto 9	2	30.88	137	164.32	18
1	P10	Produto 10	38	115.71	27.77	87.7	19
1	P11	Produto 11	154	147.99	29.39	44.95	1
1	P12	Produto 12	78	32.47	64.63	276.58	2
1	P13	Produto 13	79	194.04	58.3	99.05	3
1	P14	Produto 14	9	199.56	56.8	80.74	4
1	P15	Produto 15	131	101.15	107.6	29.24	5
1	P16	Produto 16	177	24.64	75.94	278.88	6
1	P17	Produto 17	105	195.63	126.25	183.92	7
1	P18	Produto 18	198	162.2	134.12	105.61	18
1	P19	Produto 19	148	184.36	121.69	234.58	19
1	P20	Produto 20	196	52.04	124.87	157.93	20



3.1 Ferramenta utilizada para importação

A importação dos dados foi realizada utilizando o **DBeaver**, com arquivos no formato CSV gerados a partir da planilha original.

O processo foi estruturado seguindo conceitos de ETL aplicados em ferramentas como Pentaho Data Integration (PDI), embora a execução tenha sido realizada manualmente via SQL e ferramentas de banco de dados.

3.2 Estrutura da planilha

A planilha base é composta por cinco abas principais:

- **venda**
  - venda_id, data_emissao, horariomov, produto_id, qtde_vendida, valor_unitario, filial_id, item, unidade_medida
- **pedido_compra**
  - pedido_id, data_pedido, item, produto_id, descricao_produto, ordem_compra, qtde_pedida, filial_id, data_entrega, qtde_entregue, qtde_pendente, preco_compra, fornecedor_id
- **entradas_mercadoria**
  - data_entrada, nro_nfe, item, produto_id, descricao_produto, ordem_compra, qtde_recebida, filial_id, custo_unitario
- **produtos_filial**
  - filial_id, produto_id, descricao, estoque, preco_unitario, preco_compra, preco_venda, idfornecedor

- **fornecedor**
    - idfornecedor, razao_social

3.3 Tratamentos Aplicados (Conversão, Validação e Normalização)

Durante a etapa de transformação, foram aplicados diversos tratamentos para garantir a consistência e qualidade dos dados.

3.3.1 Conversão de Tipos de Dados

- Conversão de valores numéricos com vírgula para ponto como separador decimal;
- Conversão de campos numéricos para tipos compatíveis (numeric e double precision);
- Conversão de identificadores representados como float (ex: 1.0) para tipo inteiro;
- Padronização de datas para o formato YYYY-MM-DD;
- Tratamento de campos textuais com remoção de espaços em branco.

3.3.2 Validações Aplicadas

- Verificação de campos obrigatórios (NOT NULL);

- Validação de integridade referencial entre tabelas;
- Identificação de registros com ordem_compra = 0;
- Detecção de duplicidade de registros na planilha;
- Análise de consistência de dados financeiros (preço de compra vs venda).

3.3.3 Normalizações Realizadas

- Correção de nomes de colunas com espaços ou inconsistências;
- Remoção de colunas duplicadas após consolidação;
- Padronização de nomes (produto_id);
- Alinhamento de tipos entre colunas relacionadas;
- Preparação dos dados para criação de chaves e relacionamentos.

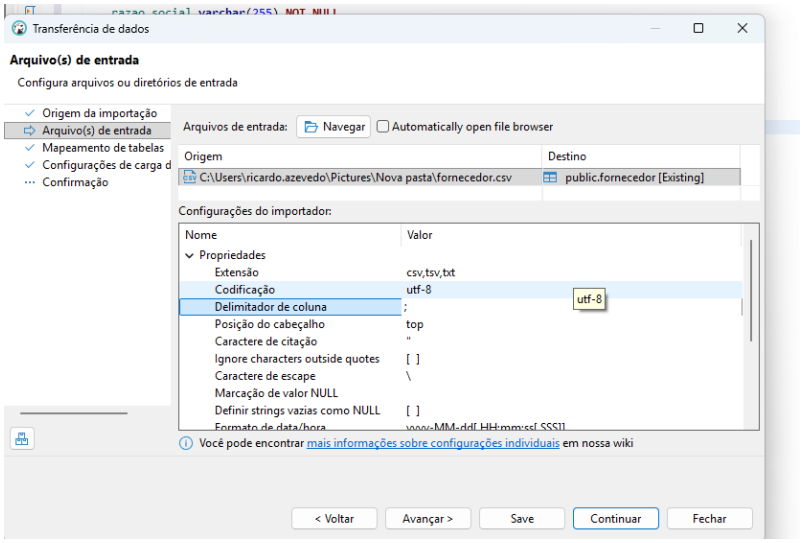
3.4 Carga dos Dados

Os dados foram carregados no PostgreSQL utilizando DBeaver com as seguintes configurações:

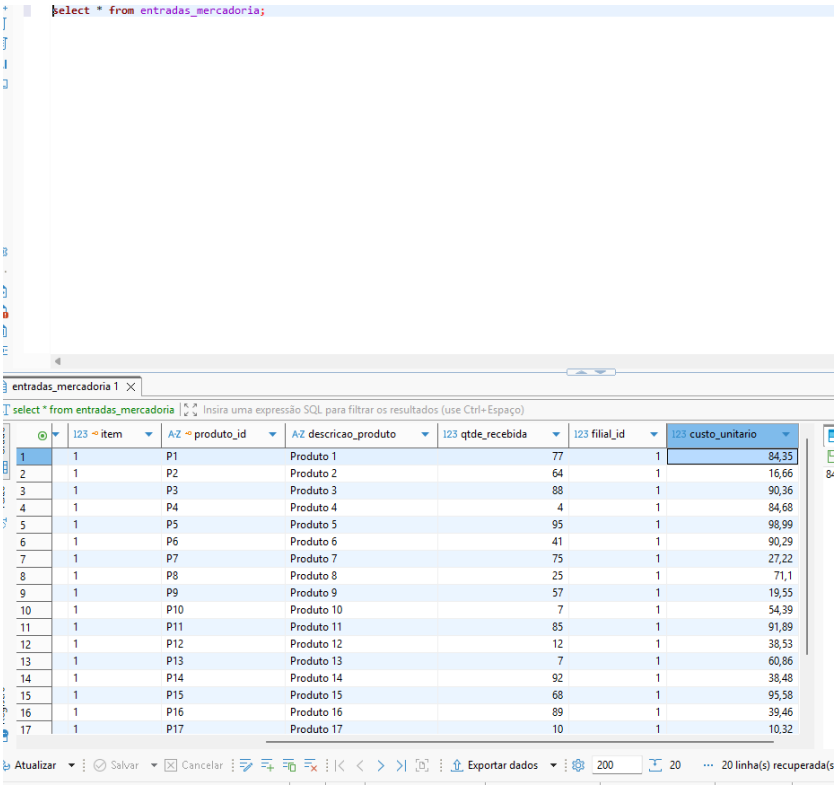
- Delimitador: ;
- Encoding: UTF-8

- Uso de cabeçalho

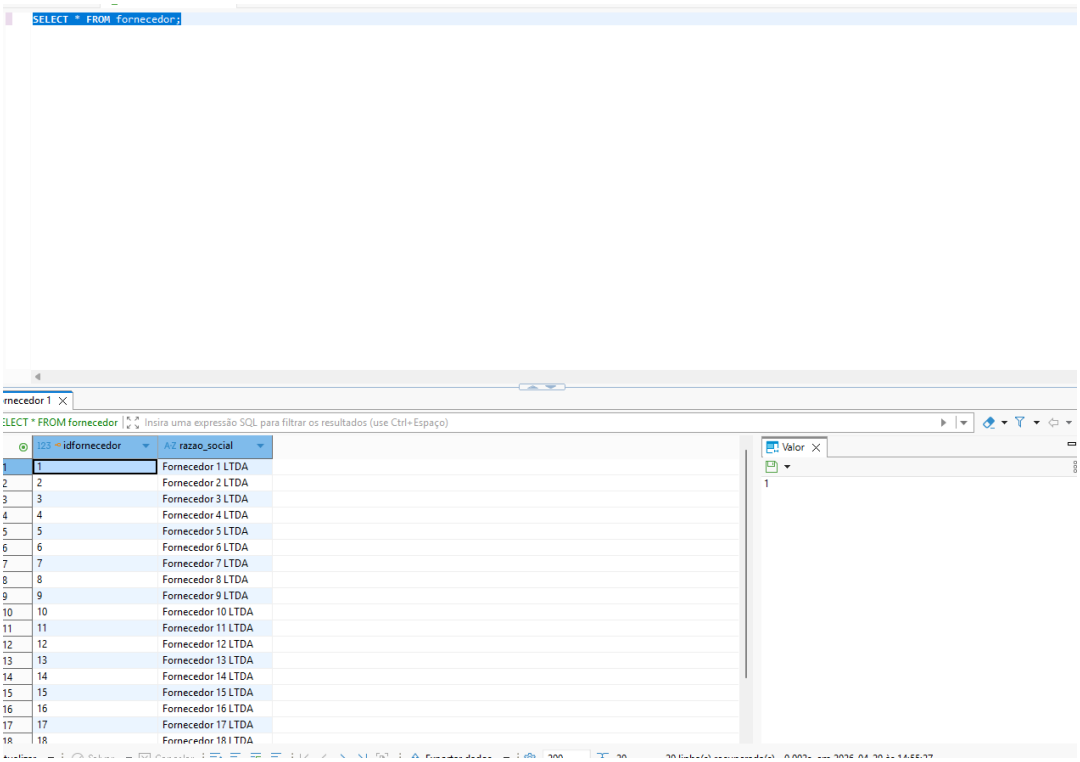
Após a carga, foram realizadas validações via SQL para garantir consistência.



Após a importação, foram realizadas consultas de validação para garantir que os dados foram corretamente persistidos e que não houve perda ou distorção de informações.



	item	produto_id	descricao_produto	qtde_recebida	filial_id	custo_unitario
1	1	P1	Produto 1	77	1	84,35
2	1	P2	Produto 2	64	1	16,66
3	1	P3	Produto 3	88	1	90,36
4	1	P4	Produto 4	4	1	84,68
5	1	P5	Produto 5	95	1	98,99
6	1	P6	Produto 6	41	1	90,29
7	1	P7	Produto 7	75	1	27,22
8	1	P8	Produto 8	25	1	71,1
9	1	P9	Produto 9	57	1	19,55
10	1	P10	Produto 10	7	1	54,39
11	1	P11	Produto 11	85	1	91,89
12	1	P12	Produto 12	12	1	38,53
13	1	P13	Produto 13	7	1	60,86
14	1	P14	Produto 14	92	1	38,48
15	1	P15	Produto 15	68	1	95,58
16	1	P16	Produto 16	89	1	39,46
17	1	P17	Produto 17	10	1	10,32



---

4. Modelagem e Integridade dos Dados

Após a carga, foram aplicadas regras de integridade no banco de dados, incluindo:

- Definição e correção de chaves primárias;
- Criação de chaves estrangeiras para garantir relacionamento entre tabelas;
- Ajustes estruturais para manter consistência entre os dados.

Essas ações asseguram que o banco de dados esteja preparado para operações analíticas e consultas complexas.

---

5. Consultas Analíticas

Foram desenvolvidas consultas SQL com o objetivo de extrair informações relevantes a partir dos dados integrados.

5.1 Análise de vendas por produto

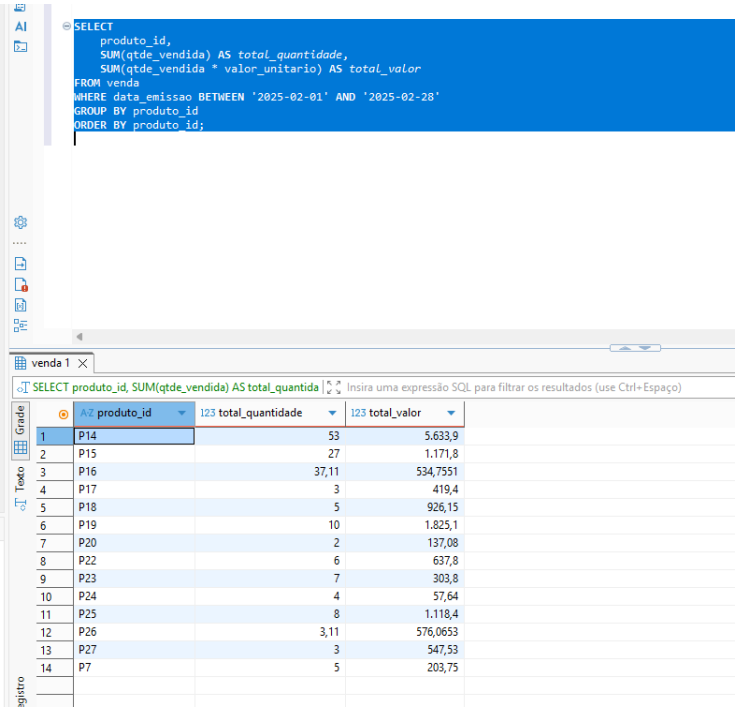
Foi realizada uma agregação das vendas por produto em um período específico, permitindo identificar volume e valor total comercializado.

5.2 Identificação de pendências em pedidos

Foram identificados pedidos onde a quantidade entregue é inferior à quantidade solicitada, evidenciando possíveis atrasos ou inconsistências no processo logístico.

5.3 Integração entre pedidos e entradas

Foi implementada uma consulta com JOIN entre pedidos de compra e entradas de mercadoria, permitindo cruzar informações e analisar o fluxo completo de aquisição de produtos.



```sql
SELECT
    produto_id,
    SUM(qtde_vendida) AS total_quantidade,
    SUM(qtde_vendida * valor_unitario) AS total_valor
FROM venda
WHERE data_emissao BETWEEN '2025-02-01' AND '2025-02-28'
GROUP BY produto_id
ORDER BY produto_id;
```

venda 1

SELECT produto_id, SUM(qtde_vendida) AS total_quantida | Insira uma expressão SQL para filtrar os resultados (use Ctrl+Espaço)

	produto_id	total_quantidade	total_valor
1	P14	53	5.633,9
2	P15	27	1.171,8
3	P16	37,11	534,7551
4	P17	3	419,4
5	P18	5	926,15
6	P19	10	1.825,1
7	P20	2	137,08
8	P22	6	637,8
9	P23	7	303,8
10	P24	4	57,64
11	P25	8	1.118,4
12	P26	3,11	576,0653
13	P27	3	547,53
14	P7	5	203,75

```sql
SELECT
    pc.produto_id,
    SUM(pc.qtde_pedida) AS total_pedido,
    SUM(pc.qtde_entregue) AS total_entregue,
    COALESCE(em.total_recebido, 0) AS total_recebido
FROM pedido_compra pc
LEFT JOIN (
    SELECT
        produto_id,
        SUM(qtde_recebida) AS total_recebido
    FROM entradas_mercadoria
    GROUP BY produto_id
) em
ON pc.produto_id = em.produto_id
GROUP BY pc.produto_id, em.total_recebido
ORDER BY pc.produto_id;
```

pedido_compra 1

SELECT pc.produto_id, SUM(pc.qtde_pedida) AS total_ped

	produto_id	total_pedido	total_entregue	total_recebido
1	P1	96	10	77
2	P10	49	4	7
3	P11	24	12	85
4	P12	103	48	12
5	P13	103	91	7
6	P14	102	27	92
7	P15	53	1	68
8	P16	93	51	89
9	P17	26	7	10
10	P18	66	17	48
11	P19	23	0	64
12	P2	14	7	64
13	P20	27	0	6
14	P3	12	2	88
15	P4	27	3	4
16	P5	35	12	95
17	P6	98	55	41
18	P7	34	29	75
19	P8	29	24	25
20	P9	57	34	57

---

6. Transformações de Dados para Apresentação

Para atender aos requisitos de negócio, foram aplicadas transformações adicionais:

- Concatenação de campos (produto_id + descrição) para melhor legibilidade;
- Conversão de datas para o formato DD/MM/YYYY;
- Aplicação de filtros para identificar produtos com maior volume de requisições;
- Uso de funções de agregação para consolidação de dados.

Essas transformações permitem uma visão mais clara e orientada à tomada de decisão.

```sql
-- Query com dados repetidos ->
SELECT
    pc.produto_id || ' - ' || pc.descricao_produto AS produto,
    SUM(pc.qtde_pedida) AS qtde_requisitada,
    TO_CHAR(pc.data_pedido, 'DD/MM/YYYY') AS data_solicitacao
FROM pedido_compra pc
GROUP BY pc.produto_id, pc.descricao_produto, pc.data_pedido
HAVING SUM(pc.qtde_pedida) > 10
ORDER BY pc.produto_id;
```

Resultados 1

SELECT pc.produto_id || ' - ' || pc.descricao_produto AS prc | Insira uma expressão SQL para filtrar os resultados (use Ctrl+Espaço)

	produto	qtde_requisitada	data_solicitacao
1	P1 - Produto 1	96	02/01/2025
2	P10 - Produto 10	49	09/01/2025
3	P11 - Produto 11	24	22/02/2025
4	P12 - Produto 12	103	25/02/2025
5	P13 - Produto 13	103	23/02/2025
6	P14 - Produto 14	102	21/01/2025
7	P15 - Produto 15	53	04/02/2025
8	P16 - Produto 16	93	27/02/2025
9	P17 - Produto 17	26	08/01/2025
10	P18 - Produto 18	66	17/02/2025
11	P19 - Produto 19	23	19/02/2025
12	P2 - Produto 2	14	07/01/2025
13	P20 - Produto 20	27	10/02/2025
14	P3 - Produto 3	12	05/01/2025
15	P4 - Produto 4	27	22/01/2025
16	P5 - Produto 5	35	28/01/2025
17	P6 - Produto 6	98	22/02/2025
18	P7 - Produto 7	34	01/03/2025
19	P8 - Produto 8	29	02/02/2025
20	P9 - Produto 9	57	15/01/2025

---

7. Automação com Trigger

Foi implementado um mecanismo de automação utilizando trigger no PostgreSQL.

O objetivo foi garantir o preenchimento automático da coluna fornecedor_id com base no valor de idfornecedor, mantendo consistência entre os dados e evitando divergências.

A trigger foi configurada para atuar antes de operações de inserção e atualização, garantindo que a regra seja aplicada em tempo real.

```sql
ALTER TABLE produtos_filial
ADD COLUMN fornecedor_id INT;

CREATE OR REPLACE FUNCTION fn_set_fornecedor_id()
RETURNS TRIGGER AS $$
BEGIN
    NEW.fornecedor_id := NEW.idfornecedor;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;


CREATE TRIGGER trg_set_fornecedor_id
BEFORE INSERT ON produtos_filial
FOR EACH ROW
EXECUTE FUNCTION fn_set_fornecedor_id();
```

---

8. Desafios Encontrados

Durante o desenvolvimento, foram enfrentados alguns desafios relevantes:

- Erros de importação causados por inconsistências no CSV;
- Problemas de tipagem de dados entre colunas relacionadas;
- Presença de colunas duplicadas com nomes semelhantes;
- Necessidade de ajustes estruturais no banco para garantir integridade;
- Dados inconsistentes que exigiram tratamento manual e validação.

A resolução desses problemas contribuiu para a robustez da solução final.

---

9. Conclusão

O processo desenvolvido contemplou todas as etapas de integração de dados, desde a análise inicial até a implementação de soluções automatizadas no banco de dados.

A abordagem adotada permitiu garantir:

- Qualidade e consistência dos dados;
- Integridade relacional;
- Capacidade de análise através de consultas SQL;
- Automação de regras de negócio via trigger.

O resultado final simula um cenário real de implantação de soluções de dados, alinhado às práticas utilizadas em ambientes corporativos.